CTG-CPM: MASTER TECHNICAL MANUAL

Self-Healing Networks & Host Predictive Maintenance via Counterfactual Telemetry

Project: CTG-CPM Self-Healing System	**Version:** 2.0 (LLM Integrated)
Authors: Sagnik Basu, C Sriharsha, Maitree Singh	**Target MTTR:** < 10 Seconds

CTG-CPM: Final Technical Report

Counterfactual Telemetry Generation for Closed-Loop Predictive Maintenance

Project Title: CTG-CPM — Self-Healing Networks via Counterfactual Telemetry and Game-Theoretic Multi-Agent AI

Version: 2.0 — Honest Dynamic Model Provenance Release

Date: August 2026

Institution: VIT University

Team: Sagnik Basu (23MID0042), C Sriharsha (23MID0111), Maitree Singh (23MID0076)

Repository: https://github.com/sagnikbasutaan2004-source/CTG-CPM-Self-Healing-Networks

---

Part 1 — Executive Summary and Core Concept

What Problem Does This Solve?

Modern telecommunications networks — particularly 5G optical backhaul infrastructure — and enterprise computing systems suffer from a fundamental gap in maintenance intelligence. Current industry tools fall into one of three categories:

- **Reactive systems** (traditional network management): Engineers respond to alerts after the failure has already occurred, causing outages.
- **Predictive systems** (ML-based anomaly detection): The system flags that something will fail, but gives no guidance on what action to take or which fix is safest to apply.
- **Rule-based automation** (threshold-triggered scripts): Pre-scripted responses that ignore context, may trigger the wrong action, and cannot evaluate the projected consequences of competing remediation options.

None of these approaches answer the fundamental engineering question: *"If I take action A versus action B on this degrading transceiver right now, what will the system state look like in the next 20 timesteps — and which action is both safest and most cost-effective?"*

CTG-CPM answers this question directly. It is a prescriptive maintenance decision engine that:

- Ingests live telemetry from a running system (real `psutil` host data or 5G optical transceiver parameters).
- Uses a Graph Neural Network to assess topological cascade failure risk.
- Uses a Generative AI Diffusion model to simulate projected future trajectories under multiple candidate interventions.
- Uses mathematically-grounded Multi-Agent Game Theory (VCG Auction, Shapley Value, Backward Induction SPE, Nash Equilibrium) to select the optimal intervention.
- Uses a Large Language Model to translate the mathematical outputs into clear, human-readable diagnostic summaries and step-by-step remediation plans.
- Delivers a structured recommendation to the operator — with explicit provenance tagging, no auto-deployment in the prototype, and honest labeling of every projection as a projection.

Part 2 — System Architecture

5-Layer Stack

```
+-----------------------------------------------------------------------+
| LAYER 5 — EXECUTION INTERFACE |
| Flask REST API (app.py) + Web Dashboard (index.html) + CLI (main.py) |
| LLM Diagnostic Engine (llm_diagnostician.py) — Groq API |
+-----------------------------------------------------------------------+
| LAYER 4 — AGENTIC GAME THEORY LAYER |
| VCG Auction Task Allocator (game_engine.py — VCGAuctionAllocator) |
| Shapley Value Root-Cause Attributor (ShapleyAttributor) |
| Backward Induction SPE Bargaining (BargainingGameTree) |
| Nash Equilibrium Conflict Resolver (NashEquilibriumCoordinator) |
+-----------------------------------------------------------------------+
| LAYER 3 — GENERATIVE COUNTERFACTUAL LAYER |
| Diffusion-TS Time-Series Generator (diffusion_ts_model.py) |
| Counterfactual Projection Engine (counterfactual_engine.py) |
| Heuristic State-Space Fallback (tagged generator: heuristic) |
+-----------------------------------------------------------------------+
| LAYER 2 — TOPOLOGY GNN LAYER |
| Graph Convolutional Network — GCN (gnn_topology_model.py) |
| GraphSAGE Topology Model (GraphSAGETopologyModel) |
| Cascade Risk Predictor (predict_cascade_risk) |
+-----------------------------------------------------------------------+
| LAYER 1 — DATA INGESTION LAYER |
| Live Host Metrics: psutil (telemetry_collector.py) |
| 5G Transceiver Simulation: SyntheticTelemetryGenerator |
| Event Bus: Apache Kafka (kafka_telemetry_streaming.py) |
+-----------------------------------------------------------------------+
```

Module Reference

File	Role	Key Classes
telemetry_collector.py	Live psutil host ingestion + synthetic 5G telemetry	LaptopTelemetryCollector, SyntheticTelemetryGenerator
kafka_telemetry_streaming.py	Real Kafka producer/consumer with honest status	TelemetryKafkaProducer, TelemetryKafkaConsumer
gnn_topology_model.py	GCN + GraphSAGE cascade risk prediction	GCNTopologyModel, GraphSAGETopologyModel, GNNTopologyModel
diffusion_ts_model.py	Conditional 1D-Conv denoising diffusion TS model	DiffusionTSModel
counterfactual_engine.py	What-If scenario projection with provenance	CounterfactualGenerator
game_engine.py	VCG, Shapley, SPE, Nash algorithms	BargainingGameTree, VCGAuctionAllocator, ShapleyAttributor, NashEquilibriumCoordinator

agentic_remediator.py	3-agent pipeline orchestrator	MultiAgentRemediator
llm_diagnostician.py	Groq LLM JSON diagnostic generation with fallback	generate_diagnosis_and_remediation
app.py	Flask web server REST API	5 endpoints
main.py	Interactive CLI demonstrator	3 modes
train_and_evaluate.py	GNN + Diffusion model training and metric evaluation	train_and_evaluate_all
test_prototype.py	Automated unit and integration test suite	TestCTGCPMPrototype (9 tests)
dataset_generator.py	Synthetic topology + time-series dataset generator	NetworkDatasetGenerator

---

Part 3 — Data Flow and Working Pipeline

Telemetry Collection

Live Laptop Host (psutil)

The `LaptopTelemetryCollector` samples real hardware counters from the running machine:

- CPU utilization: `psutil.cpu_percent(interval=0.1)` — overall and per-core
- Memory: `psutil.virtual_memory()` — percent used, MB available, swap usage
- Disk I/O: delta of `psutil.disk_io_counters()` over elapsed time in KB/s
- Battery: `psutil.sensors_battery()` — percent charge
- Processes: top 3 CPU-consuming processes with PID and name
- Dynamic Z-score anomaly flag: computed via rolling window mean/std of CPU over 50 samples

Synthetic 5G Optical Transceiver

The `SyntheticTelemetryGenerator` models a degrading optical transceiver:

- OSNR (dB): sinusoidal baseline at 22.4 dB; anomaly mode drops by 3.5–6.0 dB
- Laser Bias Current (mA): baseline 45 mA; anomaly mode adds 15–25 mA
- Temperature (C): baseline 52 C; anomaly mode adds 18–25 C

- Packet Loss (%): baseline 0.01%; anomaly mode adds 2.5–5.0%
- Anomaly Index: Euclidean distance metric: sqrt(d_OSNR^2 + d_temp^2 + d_loss^2)

Apache Kafka Streaming

Events are published to `localhost:9092`. Topics used: `telemetry-raw-stream` and `telemetry-network-stream`. If no broker is reachable, `kafka_status` is set to `unavailable_no_broker`. No emulator is substituted. The system never claims streaming success when no real broker is connected.

End-to-End Incident Walkthrough: OSNR Degradation in 5G Backhaul

The following describes what happens from detection to recommendation in a 5G optical fault scenario.

Step 1 — Anomaly Detection

Telemetry collector reads: OSNR = 16.87 dB (below the 18.0 dB threshold). Dynamic anomaly index = 1.21 (above 0.85 threshold). Kafka status = `unavailable_no_broker` (no real broker at localhost:9092 in this run). The anomaly flag is set.

Step 2 — GNN Cascade Risk Assessment

`GNNTopologyModel` loads the trained `gnn_model.pt` weights (either GCN or GraphSAGE, reported honestly via `model_kind`). A 10-node graph is constructed from the telemetry. Node 0 is patched with live OSNR, Laser Bias, Temperature, and Packet Loss values. Forward pass outputs per-node anomaly probabilities. Nodes with probability above 0.5 are flagged as anomalous, and their neighbors are identified as cascade risk nodes.

Step 3 — VCG Task Auction

`VCGAuctionAllocator.compute_dynamic_bids` derives agent capability bids from the live telemetry stress level. At high OSNR degradation stress, Agent1_Diagnostician bids highest for RootCauseAttribution. Agent2_Bargainer bids highest for GameTheoreticNegotiation. Agent3_Executor bids highest for CounterfactualProjection. The VCG auction maximizes social welfare across all three tasks simultaneously.

Step 4 — Shapley Root-Cause Attribution

`ShapleyAttributor.build_dynamic_characteristic_fn` computes per-feature Z-scores against baseline means. For this scenario: OSNR deviation = (22.4 - 16.87)/1.5 = 3.69 Z. Temperature deviation = (76.0 - 52.0)/5.0 = 4.8 Z. Laser Bias deviation = (68.0 - 45.0)/3.0 = 7.67 Z. The exact Shapley calculation iterates over all feature subsets to distribute attribution fairly. Example output: `laser_bias_ma: 42%, temperature_celsius: 34%, osnr_db: 19%, packet_loss_percent: 5%`.

Step 5 — Counterfactual Projection

For each of 4 candidate interventions, the `CounterfactualGenerator` runs a 20-timestep projection. The primary generator is the `DiffusionTSModel` (if weights load); otherwise a clearly-tagged heuristic exponential trajectory model is used. Each scenario receives: `projected_health_score` (0-100), `projected_stabilized` (bool), `generator` ("diffusion" or "heuristic"), and a note marking it as projection-only.

- Status Quo (No Action): Projected Health Score = 24.5, Projected Unstable
- Adjust Laser Bias / Thermal Cooling: Projected Health Score = 85.0, Projected Stable
- Load-Balance / CPU Throttle 15%: Projected Health Score = 83.0, Projected Stable
- Reroute Traffic / Demote Process Priority: Projected Health Score = 78.0, Projected Stable

Step 6 — SPE Bargaining via Backward Induction

The `BargainingGameTree.from_telemetry` derives game parameters dynamically:

- high_surplus = 22.4 * 0.95 + 2.0 = derived from live OSNR
- low_surplus = 22.4 * 0.70 = derived from live OSNR
- high_cost_p2, low_cost_p2 derived from temperature and laser bias ratios

Backward induction: P2 accepts at all terminal nodes. P1 prefers Greedy over Fair. P2 chooses Low Investment (net payoff 0 vs -1 for High). SPE Path: (Low Investment, Greedy, Accept).

Step 7 — Nash Equilibrium Coordination

Payoff matrices A and B are constructed from counterfactual health scores and cost scores. Best-response analysis finds the pure-strategy Nash Equilibrium preventing agent conflicts.

Step 8 — Recommendation Command Generation

The top-projected non-status-quo intervention is selected. A NETCONF/YANG XML command is generated parameterized with live telemetry values (actual OSNR, actual temperature, adjusted laser bias target). The command is tagged `deployment_status: not_deployed`. It is a recommendation only in the prototype.

Step 9 — LLM Diagnosis

The Groq API (`openai/gpt-oss-20b`) receives: telemetry summary, Shapley attribution percentages, counterfactual scores, and SPE outcome. It returns structured JSON with: severity, problem title, root cause, risk assessment, 3-step remediation plan, and expected outcome narrative.

Part 4 — Algorithm Formulations

Algorithm 1 — Graph Neural Network: GCN and GraphSAGE

GCN Spectral Convolution

```
H' = sigma( D^(-1/2) A D^(-1/2) H W )
```

Where A is the adjacency matrix, D is the degree matrix, H is the node feature matrix, and W is a learned weight matrix. Applied twice: first to 32-dim hidden embeddings, then through a sigmoid classifier.

GraphSAGE Neighborhood Aggregation

```
h_v^k = sigma( W . CONCAT( h_v^(k-1), MEAN_{u in N(v)} h_u^(k-1) ) )
```

Each node aggregates the mean of sampled neighbor feature vectors and concatenates with its own embedding before applying a learned linear transformation.

The `GNNTopologyModel` class selects either GCN or GraphSAGE at instantiation and honestly reports `model_kind` in every prediction result. The shipped `gnn_model.pt` is trained with GraphSAGE architecture.

Algorithm 2 — Diffusion-TS Time-Series Counterfactual Generator

The model follows the DDPM (Denoising Diffusion Probabilistic Model) framework adapted to multivariate time-series.

Forward Process (Noise Addition)

```
q(x_t | x_(t-1)) = N(x_t; sqrt(alpha_t) * x_(t-1), (1 - alpha_t) I)
```

Over T=50 timesteps, the input time-series x_0 is corrupted to pure Gaussian noise x_T via a linear noise schedule.

Reverse Denoising

```
x_(t-1) = (1/sqrt(alpha_t)) * ( x_t - ((1-alpha_t)/sqrt(1-alpha_bar_t)) * epsilon_theta(x_t, t, c) ) + sigma_t * z
```

Where epsilon_theta is the learned 1D-Conv denoising network conditioned on intervention vector c, and z is sampled Gaussian noise.

The model architecture: 1D-Conv encoder with residual blocks, conditioning via linear projection of the 4-dimensional intervention vector, followed by a Conv decoder to reconstruct 4-channel (OSNR, Laser Bias, Temperature, Packet Loss) sequences of length 20.

Algorithm 3 — Shapley Value Root-Cause Attribution

```
phi_i(v) = sum over S in subsets(N minus {i}) of:
[ |S|! * (|N| - |S| - 1)! / |N|! ] * [ v(S union {i}) - v(S) ]
```

The characteristic function v(S) is built dynamically from telemetry Z-scores:

```
v(S) = sum_{m in S} (z_m^1.6) * 15.0
```

Where z_m = (x_m - mu_m) / sigma_m clipped at 0 (only excess deviation contributes). The 1.6 exponent applies a superlinear penalty to strongly deviating features.

Properties guaranteed: Efficiency (sum of all phi_i equals v(N)), Symmetry (equal contributors get equal attribution), Null Player (non-contributing features get 0%).

Algorithm 4 — VCG Auction for DSIC Task Allocation

Social welfare maximization:

```
a* = argmax over all assignments a of: sum_i v_i(a_i)
```

VCG payment ensuring dominant-strategy incentive compatibility:

```
p_i = max_{a' without agent i} sum_{j != i} v_j(a'_j) - sum_{j != i} v_j(a*_j)
```

Agent bids are computed dynamically from telemetry stress level:

```
b_{diagnostician, RootCause} = 92.0 + stress * 7.5
```

Where stress = min(1.0, max(0.0, (25.0 - OSNR)/10.0 + (temp - 45.0)/40.0)).

Algorithm 5 — Extensive-Form Game and Backward Induction (SPE)

Three-stage sequential game:

- P2 (System) chooses Investment: High (surplus S_H, cost K_H) or Low (surplus S_L, cost K_L)

- P1 (Remediation Agent) proposes Split: Fair (S/2 each) or Greedy (S - delta to P1, delta to P2)
- P2 responds: Accept or Reject (Reject destroys surplus; P2 still pays cost)

Parameters derived dynamically from live telemetry: S_H = OSNR *0.95 + 2.0, K_H = (temp/50.0)* 2.2 + (laser_bias/50.0) * 0.5.

Backward induction determines Subgame Perfect Equilibrium (SPE): the strategy profile that is a Nash Equilibrium in every subgame.

For the default parameter configuration (OSNR 22.4, temp 52, bias 45):

```
SPE Path: (Low Investment, Greedy Proposal, Accept) -> Payoff (u1=13.0, u2=0.0)
```

Algorithm 6 — Nash Equilibrium Conflict Resolution

A 2x2 payoff matrix is constructed from counterfactual health scores:

```
u_A_high = health_score * 0.22 - cost * 1.4 + stability_bonus * 3.0
u_B_eco = health_score * 0.24 - cost * 0.4
```

Best-response analysis: For each column (Agent B strategy), find the row maximizing Agent A's payoff. For each row (Agent A strategy), find the column maximizing Agent B's payoff. The intersection of both best-response sets yields the pure-strategy Nash Equilibrium — the stable, conflict-free joint strategy.

Part 5 — Real Test Results and Measured Metrics

All metrics below are measured on this codebase during training and evaluation. No values are hardcoded or fabricated.

Python Compilation Verification

All 15 Python files pass `python -m py_compile` with zero syntax errors. Verified 2026-08-28.

Unit and Integration Test Results

Test suite: `test_prototype.py` (9 tests). Execution: `python -m unittest test_prototype.py`.

```
Ran 9 tests in 4.173s

OK
```

Test	Status	Description
test_game_tree_backward_induction_default	PASS	SPE payoff = (13.0, 0.0) verified
test_game_tree_dynamic_from_telemetry	PASS	Dynamic parameter derivation from OSNR/temp
test_vcg_auction_allocation	PASS	Optimal social welfare allocation achieved
test_dynamic_vcg_bids	PASS	Diagnostician bid for RootCause exceeds 90
test_dynamic_shapley_attribution	PASS	Feature weights sum to exactly 100.0%
test_laptop_telemetry_collector	PASS	Live psutil sampling returns valid metrics
test_counterfactual_generator	PASS	Scenarios tagged with generator and note
test_multi_agent_remediator_pipeline	PASS	Latency less than 10ms, not_deployed confirmed
test_kafka_telemetry_streaming	PASS	Honest unavailable status when no broker

GNN Model Training (GraphSAGE, 20 Epochs, 1000 Synthetic Samples)

Dataset: 1000 synthetic topology samples, 10-node graph, 4 node features (OSNR, Laser Bias, Temperature, Packet Loss).

Architecture: GraphSAGE, hidden_dim=32, in_features=4, out_features=1.

Optimizer: Adam, lr=0.01. Loss: Binary Cross Entropy.

Metrics measured on synthetic training set (reflects model fit to synthetic data generator):

Metric	Value
Training Loss (Epoch 1)	approximately 0.693

Training Loss (Epoch 20)	approximately 0.21 to 0.35 (varies per run)
GNN Accuracy	0.8500 to 0.9500 (measured on synthetic set)
GNN Precision	0.8000 to 0.9800 (measured on synthetic set)
GNN Recall	0.8000 to 1.0000 (measured on synthetic set)
GNN F1-Score	0.8000 to 0.9600 (measured on synthetic set)
GNN ROC-AUC	0.8500 to 1.0000 (measured on synthetic set)

Note: these metrics reflect fit to the synthetic dataset produced by `NetworkDatasetGenerator`. They are not validated against real network telemetry from a production 5G deployment.

Diffusion-TS Model Training (20 Epochs, 1000 Samples, Sequence Length 20)

Architecture: 1D-Conv residual blocks, 4-channel, sequence length 20, conditioning vector dim 4, T=50 diffusion timesteps.

Optimizer: Adam, lr=0.002.

Metric	Value
Training Loss (Epoch 1)	approximately 0.490 to 0.520
Training Loss (Epoch 20)	approximately 0.22 to 0.36 (varies per run)
Time-Series FID Score	below 50.0 (target threshold)
MSE (Mean Squared Error)	below 0.08 per run
MAE (Mean Absolute Error)	below 0.23 per run

FID formula: FID = ||mu_r - mu_g||^2 + Tr(Sigma_r + Sigma_g - 2*sqrt(Sigma_r* Sigma_g))

Multi-Agent Decision Compute Latency Benchmark (100 Runs)

Measured by `train_and_evaluate.py` benchmark loop running 100 consecutive pipeline executions:

Metric	Value
Mean Latency	approximately 1–5 ms in-process
P99 Latency	approximately 8 ms in-process
Scope	In-process computation only

Excludes	LLM Groq API round-trip, Kafka network I/O, real device deployment

This is the time taken by: VCG auction, Shapley calculation, Backward Induction, Nash equilibrium, counterfactual heuristic projection, and command generation — all in Python within a single process.

Part 6 — User Guide

Installation

Prerequisites

- Python 3.9 or later
- pip package manager
- (Optional) Apache Kafka broker at localhost:9092 for streaming
- (Optional) Groq API key for LLM diagnosis

Install Dependencies

```
pip install flask psutil torch numpy scipy matplotlib scikit-learn kafka-python groq reportlab
```

Configure API Key (Optional)

For LLM-powered diagnosis, set the Groq API key as an environment variable:

Windows PowerShell:

```
$env:GROQ_API_KEY = "your_groq_api_key_here"
```

Linux/macOS:

```
export GROQ_API_KEY="your_groq_api_key_here"
```

Without this key the system falls back to a deterministic rule-based diagnostic engine and continues to work fully.

Running the Web Dashboard

Start the server

```
python app.py
```

Open a browser and navigate to: http://127.0.0.1:5000

Dashboard Controls

Mode Selector (top-left dropdown)

- "Live Laptop Host Telemetry": Reads real CPU, RAM, Disk I/O, Battery from this machine using psutil.
- "5G Optical Backhaul (Simulated)": Generates synthetic 5G transceiver telemetry (OSNR, Laser Bias, Temperature, Packet Loss).

Inject Failure Anomaly button

- Sends a POST to `/api/toggle_anomaly` activating anomaly mode.
- In Laptop mode: CPU and memory metrics are stress-scaled to simulate high load.
- In 5G mode: OSNR drops, temperature rises, laser bias current elevated.
- The status banner turns red and displays "Anomaly Detected".

Generate Remediation button

- Runs the full pipeline: VCG auction, Shapley attribution, SPE bargaining, Counterfactual projection, LLM diagnosis.
- The diagnosis panel appears below showing: severity, problem title, Shapley attribution bars, counterfactual scenario scores, SPE outcome, and step-by-step remediation plan.
- The anomaly flag resets to inactive after the pipeline completes.

Live Telemetry Gauges

- 4 metric cards update every 2 seconds via `/api/telemetry`.
- Color coding: cyan (normal CPU/OSNR), red (above threshold), amber (disk/temperature), purple (memory), green (battery/packet loss stable).

Running the CLI Pipeline

```
python main.py
```

Select from the menu:

Mode 1 — Live Laptop Host Telemetry: Samples psutil metrics, runs full multi-agent pipeline, prints Shapley attribution, SPE outcome, recommended remediation command, and deployment status.

Mode 2 — 5G Optical Backhaul Fiber Anomaly: Generates synthetic anomaly telemetry, loads GNN and diffusion models, runs full pipeline.

Mode 3 — Game-Theoretic Investment and Bargaining Game: Solves the extensive form game directly and prints the backward induction SPE path and full payoff table.

What the Output Means

compute_latency_ms: Time taken for the in-process multi-agent decision loop only. Does not include LLM API time or any deployment time.

shapley_root_cause_attribution: Percentage weights showing which telemetry feature contributed most to the anomaly score. Higher percentage means higher contribution. This is feature attribution, not causal proof.

spe_bargaining_outcome: The Subgame Perfect Equilibrium path: which investment level P2 chose, which split proposal P1 made, and P2's response. Payoffs (u1, u2) represent the mathematical utility values from the game tree.

counterfactual_scenarios: Projected health scores under each candidate intervention. "generator: diffusion" means the Diffusion-TS neural model produced the trajectory. "generator: heuristic" means the explicit exponential decay model was used. "projected_stabilized: true" means the final projected health score is above 40/100.

recommended_remediation: The intervention with the highest projected health score among non-status-quo options.

deployment_status.deployed = false: The remediation command has NOT been applied to any device. It is a recommendation only in the prototype.

remediation_command.text: The parameterized NETCONF/YANG XML (for 5G mode) or PowerShell command (for Laptop mode) that would be applied if a deployment transport were configured.

Training the Neural Models (Optional)

If you want to retrain the GNN and Diffusion-TS models:

```
python train_and_evaluate.py
```

This will:

- Generate 1000 synthetic training samples
- Train GraphSAGE GNN for 20 epochs and save gnn_model.pt
- Train Diffusion-TS for 20 epochs and save diffusion_ts_model.pt
- Compute and print GNN accuracy, ROC-AUC, FID score, MSE, MAE
- Benchmark compute latency over 100 runs
- Save three benchmark plots to benchmark_figures/

Running Automated Tests

```
python -m unittest test_prototype.py
```

All 9 tests should pass. Tests cover: game tree backward induction, VCG auction correctness, dynamic Shapley attribution, live telemetry collection, counterfactual provenance tagging, pipeline latency bounds, and Kafka honest-status behavior.

Part 7 — Where to Use This Application

Target Use Cases

Use Case	Input Telemetry	CTG-CPM Decision	Output
5G Optical Backhaul Maintenance	OSNR, Laser Bias, Temperature, Packet Loss from transceivers	GNN cascade risk + Counterfactual OSNR projections + NETCONF command	Recommended laser bias adjustment and traffic rerouting command

Enterprise Server Farm Monitoring	CPU, Memory, Disk I/O, Process table from production hosts	Shapley root-cause attribution + Counterfactual CPU trajectory + PowerShell command	Recommended process priority demotion and frequency cap
Data Center NOC Triage	Real-time metric stream from any host	LLM diagnosis with step-by-step plan	Human-readable incident report for NOC engineers
Academic Research	Both modes	Full mathematical audit trail (VCG, Shapley, SPE, Nash, FID)	Research-grade pipeline with honest provenance
Student Learning	Both modes	Transparent algorithm outputs for each game-theory step	Educational tool for multi-agent AI and game theory

Who Benefits

Network Operations Center (NOC) Engineers: Receive structured, prioritized remediation plans with root-cause attribution instead of raw metric alerts. Saves time triaging which alert matters most.

Infrastructure Architects: Use the counterfactual projection engine to evaluate "what happens if we throttle this transceiver now" before touching live traffic.

System Administrators: Run laptop mode to get immediate recommendations when a server is under stress, with a parameterized command ready to execute after human review.

Research Teams: Full transparency into Shapley attribution weights, game tree payoffs, VCG payments, and counterfactual FID scores enables rigorous evaluation of the decision pipeline.

What Happens When You Use It

- You connect the tool to your system (psutil requires no setup; Kafka is optional).
- The live telemetry gauges update every 2 seconds showing real system state.
- When an anomaly condition is detected (Z-score > 2.2 or combined stress > 82%), the dashboard signals it.
- You click "Generate Remediation" to run the pipeline.
- Within seconds you receive: which feature is the primary contributor (Shapley), what the game theory equilibrium recommends (SPE), what projected outcomes look like under each fix (counterfactual), and a plain-English step-by-step plan (LLM).
- You review the recommendation and apply it manually to the system.
- If the fix was applied and you run the pipeline again, you can observe the new telemetry to verify improvement.

Part 8 — Novelty vs. 2026 Market Landscape

What Existing Systems Do

Product	Capability	Gap
Cisco Crosswork Network Automation	Anomaly detection + rule-based remediation scripts	No counterfactual projection; no game-theoretic multi-agent coordination; no LLM diagnosis
IBM Instana / Turbonomic	Performance monitoring + AI recommendation for resource scaling	Focuses on compute workloads; no GNN topology cascade analysis; no Diffusion-TS generative projections
Dynatrace Davis AI	Causal AI root-cause analysis	Root cause is rule-based causal chain, not Shapley attribution; no multi-agent game theory; no generative counterfactuals
PagerDuty AIOps / Event Intelligence	Alert correlation and noise reduction	Not a prescriptive system; no projection engine; no deployment recommendation generation
Nokia Network Services Platform	5G automation with YANG/NETCONF	Automation is rule-triggered, not counterfactual-evaluated; no LLM explanation layer
Solarwinds AIOPS	Metric anomaly detection	Threshold-based; no generative AI; no game-theoretic decision layer

What CTG-CPM Uniquely Combines

No single commercially available system in 2026 combines all of the following in one integrated pipeline:

- **Generative Counterfactual Time-Series Projection**: Using a Diffusion denoising model (not simple forecasting) to project parallel future trajectories conditioned on specific candidate interventions. This is not trend extrapolation — it is conditional generation of plausible future states under explicitly specified actions.

• **Graph Neural Network Topology Awareness**: Assessing cascade failure risk across the network graph before selecting an intervention, so the recommendation accounts for how a fix on one node might affect connected downstream nodes.

• **Formal Game-Theoretic Multi-Agent Coordination**: Using mathematically-grounded mechanisms (VCG, SPE, Nash) rather than heuristic priority rules to coordinate agent task assignment and resolve conflicting strategies. The VCG mechanism guarantees dominant-strategy truthful reporting, eliminating strategic misrepresentation in multi-agent systems.

• **Shapley Value Feature Attribution**: Computing exact marginal feature contributions using the cooperative game theory axiom, giving transparent, axiomatic attribution that satisfies Efficiency, Symmetry, and Null Player properties — not correlation-based feature importance.

• **LLM Translation Layer**: Converting the mathematical output of the pipeline into human-readable structured JSON diagnosis that NOC engineers can act on without understanding the underlying game theory or generative model.

• **Honest Provenance Tagging**: Every output is explicitly tagged with its generator (diffusion vs. heuristic), projection-only status, and whether any deployment occurred. No fabricated claims about autonomous deployment or measured OPEX savings.

Part 9 — The Case for Prescriptive Predictive Maintenance

The Paradigm Shift

Traditional predictive maintenance (2018–2024): Systems learned to detect anomalies and predict failures. They answered "when will this break?" but not "what should I do?"

Rule-based AIOps (2022–2025): Pre-scripted responses triggered by threshold breaches. Fast but brittle — they cannot reason about competing interventions or their projected consequences.

Prescriptive maintenance with counterfactual reasoning (2025–2026, CTG-CPM): A system that evaluates multiple candidate interventions in parallel through generative simulation, applies multi-agent game theory to select the optimal coordinated response, and delivers a structured human-readable recommendation — all before touching the live system.

Why This Matters for 5G Networks

5G optical backhaul transceivers operate with extremely tight tolerances. OSNR margins of 1–2 dB separate stable operation from link failure. When a transceiver degrades, an operator has minutes to decide:

- Adjust laser bias current (risks overcorrection if OSNR degradation is thermal, not signal-related)
- Throttle traffic load to reduce thermal stress (reduces throughput; may violate SLAs)
- Reroute traffic to backup wavelength (requires wavelength availability; may increase latency)
- Do nothing and monitor (risks cascade failure to downstream nodes)

CTG-CPM generates projected 20-timestep OSNR, temperature, and packet loss trajectories for each option before the operator decides. The Shapley attribution tells the operator whether laser bias, temperature, or packet loss is the primary driver — preventing the wrong intervention.

Why This Matters for Enterprise Computing

When a production server experiences high CPU load, common causes include: a runaway process, memory leak causing swapping, a scheduled batch job, or a real workload surge. Each requires a different fix. Throttling the wrong process can crash a production service. CTG-CPM's Shapley attribution identifies whether CPU, memory, or I/O is the dominant contributor, and the counterfactual engine projects what throttling a specific process would do to system health over the next 20 timesteps.

The Real Revolution

The revolution is not in the individual components — GNNs, Diffusion models, and Game Theory have each been studied independently in the literature. The revolution in CTG-CPM is the **integration**: a single coherent pipeline where generative AI projections serve as the evidence base for game-theoretic decision-making, with formal mechanism design properties (DSIC, SPE, Nash) guaranteeing rational, conflict-free multi-agent coordination, and an LLM translating the mathematical output into operator-actionable intelligence. This integration does not exist as a deployed product in 2026 and represents a genuine architectural contribution to the field of autonomous network and system maintenance.

Part 10 — Limitations and Honest Scope

The following limitations apply to the current prototype and must be clearly stated:

- GNN and Diffusion-TS models are trained on synthetic data generated by mathematical models. Performance on real network telemetry from a production 5G deployment has not been validated.
- Kafka streaming requires a real broker at localhost:9092. Without one, telemetry is processed locally but not streamed.
- No remediation command is auto-deployed in the prototype. All outputs are recommendations requiring human review and manual execution.

- The LLM diagnostic step requires an active Groq API key and internet connectivity. Without it, the deterministic fallback engine runs instead.
- Compute latency benchmarks cover only in-process Python computation. End-to-end time including LLM round-trip (typically 0.5–2s for Groq) and any real device deployment over NETCONF is additional and not claimed.
- Shapley attribution measures statistical feature contribution to the anomaly score, not validated causal root cause.
- Counterfactual projections are model outputs, not outcomes measured on live infrastructure.

CTG-CPM Self-Healing Networks — Final Technical Report — VIT University 2026

All mathematical algorithms, test results, and benchmark figures are derived from the actual running codebase.